

9.3 哈希表

思想：在记录的存储地址和它的关键字之间建立一个确定的对应关系；不经过比较，一次存取就能得到所查元素的查找方法。

例 30个地区的各民族人口统计表

编号	地区	总人口	汉族	回族.....
1	北京			
2	上海			
⋮	⋮			

以编号作关键字，
构造哈希函数： $H(\text{key})=\text{key}$
 $H(1)=1$
 $H(2)=2$

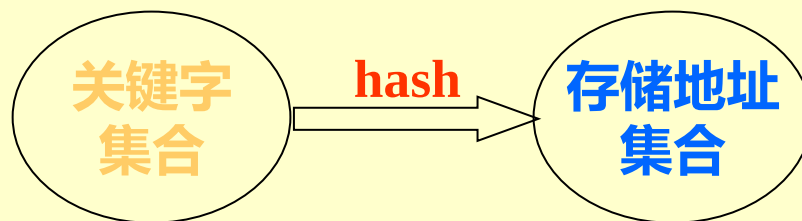
以地区作关键字，取地区
名称第一个拼音字母的序号
作哈希函数： $H(\text{Beijing})=2$
 $H(\text{Shanghai})=19$

❖ **哈希函数**——在记录的关键字与记录的存储地址之间建立的一种对应关系

□ 哈希函数是从关键字空间到存储地址空间的一种映象

—哈希函数可写成: $H(k_i) = \text{addr}(a_i)$

其中, a_i 是表中的一个元素, $\text{addr}(a_i)$ 是 a_i 的存储地址, k_i 是 a_i 的关键字



设哈希函数为: $\text{int}(K/3)+1$

构造 01、02、05、09、11、13、16、19、21、26、27、31、的散列表(哈希表) 为:

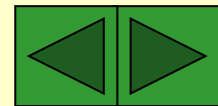
表项序号	1	2	3	4	5	6	7	8	9	10	11	12
$H(K)=\text{int}(K/3)+1$	01	05		09	13	16	19	21	26	27	31	
	02			11								

❖ **哈希表**——应用哈希函数，由记录的关键字确定记录在表中的地址，并将记录放入此地址，这样构成的表

❖ **哈希查找**——又叫散列查找，利用哈希函数进行查找的过程

特点：

- **哈希函数**是一种映象，设定灵活，只要使任何关键字的哈希函数值都落在表长允许的范围之内即可
- **冲突**： $\text{key1} \neq \text{key2}$ ，但 $H(\text{key1}) = H(\text{key2})$ 的现象叫**冲突**
- **同义词**：具有相同函数值的两个关键字，该哈希函数的**同义词**
- **哈希函数通常是一种压缩映象，所以冲突不可避免**，只能尽量减少；同时，冲突发生后，应该有处理冲突的方法



9.3.2 哈希函数的构造方法

• 直接定址法

- 构造：取关键字或关键字的某个**线性函数**作哈希地址，即 $H(\text{key})=\text{key}$ 或

$$H(\text{key})=a \cdot \text{key} + b$$

- 特点

直接定址法所得地址集合与关键字集合大小相等，不会发生冲突

- 实际中能用这种哈希函数的情况很少

例如：关键字集合为{100, 300, 500, 700, 800, 900}，选取哈希函数为

$\text{Hash}(\text{key})=\text{key}/100$ ，则存放如下：

0 1 2 3 4 5 6 7 8 9

	100		300		500		700	800	900
--	-----	--	-----	--	-----	--	-----	-----	-----

• 数字分析法

- 构造：对关键字进行分析，取关键字的若干位或其组合作哈希地址
- 适于关键字位数比哈希地址位数大，且可能出现的关键字事先知道的情况

例 有80个记录，关键字为8位十进制数，哈希地址为2位十进制数

①②③④⑤⑥⑦⑧

⋮
8 1 3 | 4 6 5 3 | 2
8 1 3 | 7 2 2 4 | 2
8 1 3 | 8 7 4 2 | 2
8 1 3 | 0 1 3 6 | 7
8 1 3 | 2 2 8 1 | 7
8 1 3 | 3 8 9 6 | 7
8 1 3 | 6 8 5 3 | 7
8 1 4 | 1 9 3 5 | 5
⋮

分析：①只取8

②只取1

③只取3、4

⑧只取2、7、5

④⑤⑥⑦数字分布近乎随机

所以：取④⑤⑥⑦任意两位或两位与另两位的叠加作哈希地址

●平方取中法

- 构造：取关键字平方后中间几位作哈希地址
- 适于不知道全部关键字情况
- 如： $K=308$, $K^2=94864$, $H(K)=486$

●折叠法

- 构造：将关键字分割成位数相同的几部分，然后取这几部分的叠加和（舍去进位）做哈希地址
- 适于关键字位数很多，且每一位上数字分布大致均匀情况

种类

- 移位叠加：将分割后的几部分低位对齐相加

例 关键字为：
0442205864，哈
希地址位数为4

$$\begin{array}{r} 5864 \\ 4220 \\ 04 \\ \hline 10088 \end{array}$$

$$H(\text{key})=0088$$

移位叠加

❖ **间界叠加**：从一端沿分割界来回折送，然后对齐相加

例 关键字为：
0442205864，哈
希地址位数为4

$$\begin{array}{r} 5\ 8\ 6\ 4 \\ 0\ 2\ 2\ 4 \\ \quad 0\ 4 \\ \hline 6\ 0\ 9\ 2 \end{array}$$

间界叠加

$$H(\text{key})=6092$$

• 除留余数法

— 构造：取关键字被某个**不大于**哈希表表长 m 的数 p 除后所得余数作哈希地址，即

$$H(\text{key})=\text{key} \bmod p, \quad p \leq m$$

— 特点

» 简单、常用，可与上述几种方法结合使用

» p 的选取很重要； p 选的不好，容易产生同义词

• 随机数法

- 构造：取关键字的随机函数值作哈希地址，即 $H(\text{key})=\text{random}(\text{key})$
- 适于关键字长度不等的情况

❖ 选取哈希函数，考虑以下因素：

- 计算哈希函数所需时间
- 关键字长度
- 哈希表长度（哈希地址范围）
- 关键字分布情况
- 记录的查找频率

哈希方法需要解决以下两个问题：

1. 构造好的哈希函数

(1) 所选函数尽可能简单，以便提高转换速度。

(2) 所选函数对关键字计算出的地址，应在哈希地址集中大致均匀分布，以减少空间浪费。

2. 制定解决冲突的方案。

9.3.3处理冲突的方法

- 开放定址法

- 方法：当冲突发生时，形成一个探查序列；沿此序列逐个地址探查，直到找到一个空位置（开放的地址），将发生冲突的记录放到该地址中，即 $H_i = (H(\text{key}) + d_i) \text{MOD } m$ ， $i=1,2,\dots,k(k \leq m-1)$

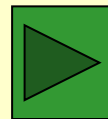
其中：H(key)——哈希函数

m——哈希表表长

d_i ——增量序列

- 分类

- » 线性探测再散列： $d_i=1,2,3,\dots,m-1$
- » 二次探测再散列： $d_i=1^2,-1^2,2^2,-2^2,3^2,\dots,\pm k^2(k \leq m/2)$
- » 伪随机探测再散列： $d_i=\text{伪随机数序列}$



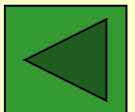
例 表长为11的哈希表中已填有关键字为17, 60, 29的记录,
 $H(\text{key}) = \text{key} \bmod 11$, 现有第4个记录, 其关键字为38,
按三种处理冲突的方法, 将它填入表中

0	1	2	3	4	5	6	7	8	9	10
			38	38	60	17	29	38		

(1) $H(38) = 38 \bmod 11 = 5$ 冲突
 $H_1 = (5+1) \bmod 11 = 6$ 冲突
 $H_2 = (5+2) \bmod 11 = 7$ 冲突
 $H_3 = (5+3) \bmod 11 = 8$ 不冲突

(2) $H(38) = 38 \bmod 11 = 5$ 冲突
 $H_1 = (5+1^2) \bmod 11 = 6$ 冲突
 $H_2 = (5-1^2) \bmod 11 = 4$ 不冲突

(3) $H(38) = 38 \bmod 11 = 5$ 冲突
设伪随机数序列为9, 则:
 $H_1 = (5+9) \bmod 11 = 3$ 不冲突



再哈希法

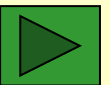
- 方法：构造若干个哈希函数，当发生冲突时，计算下一个哈希地址，即： $H_i = RH_i(\text{key})$
 $i=1,2,\dots,k$

其中：RH_i——不同的哈希函数

- 特点：计算时间增加

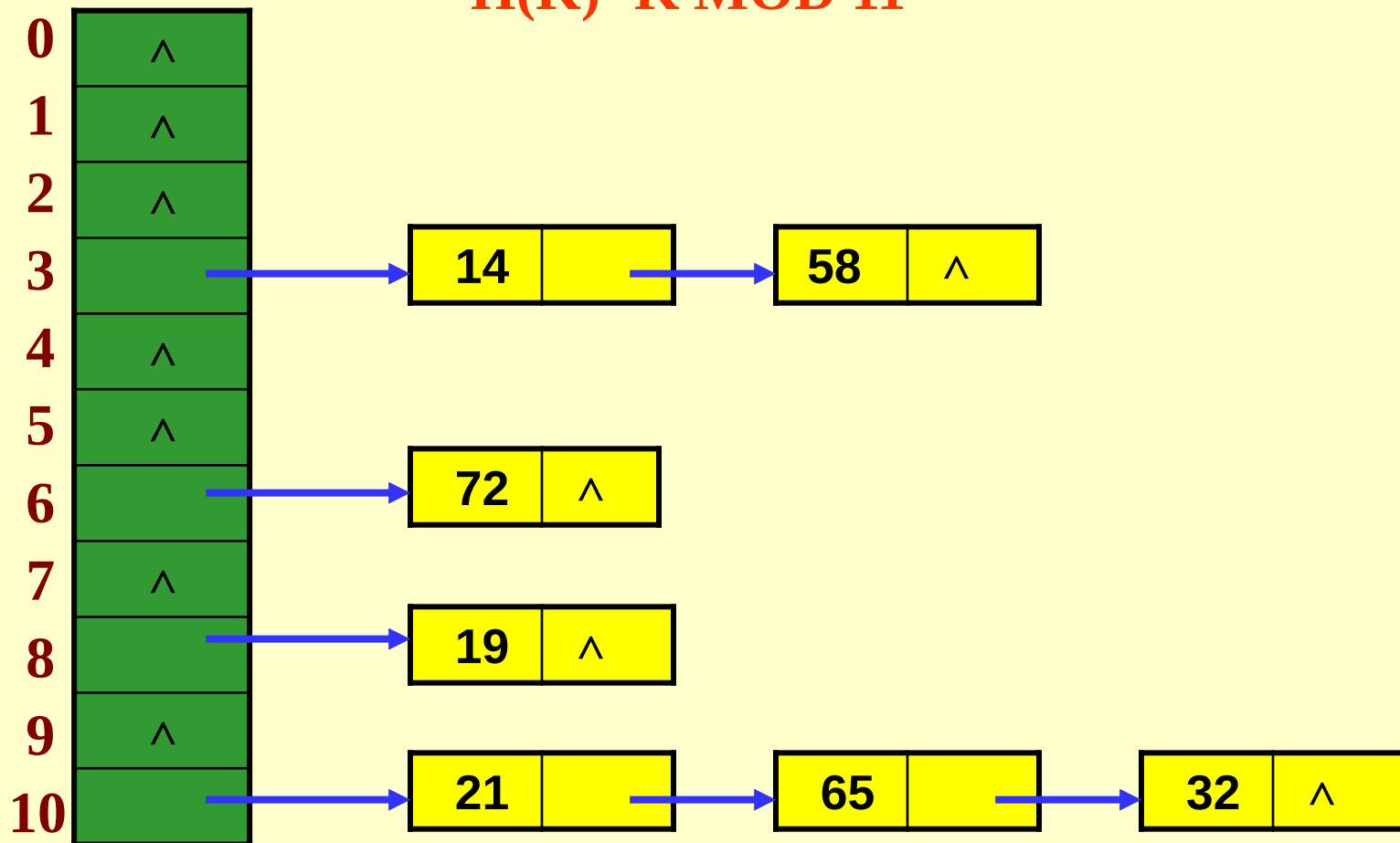
链地址法(空间索引)

- 方法：将所有关键字为同义词的记录存储在一个单链表中，并用一维数组存放头指针。
- 优点：插入、删除方便
- 缺点：占用存储空间多。



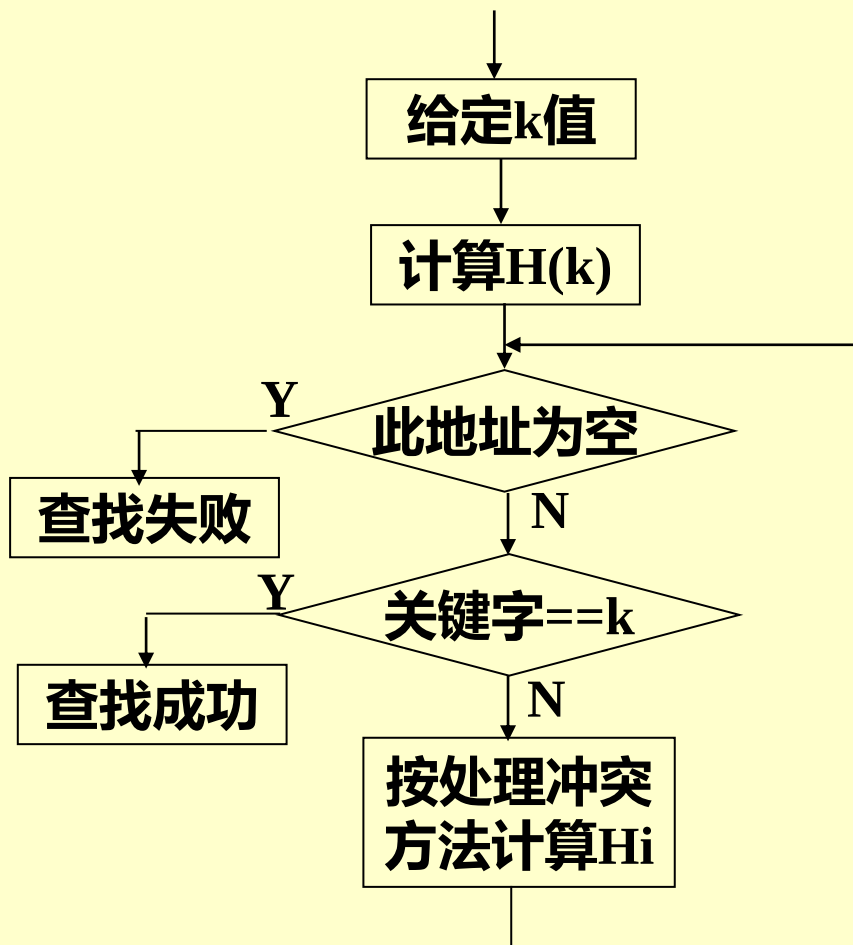
例 一组关键字为 (21, 14, 19, 58, 65, 32, 72)

$$H(K) = K \text{ MOD } 11$$



9.3.4 哈希表的查找及其分析

• 哈希查找过程



//--- 开放定址哈希表的存储结构 ---

```
#define HASH_LENGTH 11 //表长
#define HASH_M 11 //哈希函数%11
#define SUCCESS 1
#define UNSUCCESS 0

typedef int Status;

typedef struct{
    int *elem;//定义关键字为整型
    int count;//当前哈希表中的关键字个数
    int length;//当前哈希表的容量
}HashTable;
```

/*查找key在哈希表中的位置，如查找成功返回SUCCESS，否则返回UNSUCCESS。用p返回key在哈希表中的位置, c为处理冲突次数。用p也可返回key要插入的位置, c为处理冲突的次数*/

Status SearchHash(HashTable H, int key, int *p, int *c)

{ int q;

***c = 0; //重要！相当于全局变量，要重新赋初值**

***p = _____; //取得关键字的哈希地址**

**//若存储位置已有关键字(不等0)，且关键字值与key不相等则
进行冲突处理**

while (_____ != 0 && _____ != key){

***p = Collision(_____, _____); }**

if (_____ == key) return SUCCESS;

else{ return UNSUCCESS; }

}

哈希表满了而且找不到key，陷入死循环。



/*若哈希表H中无关键字key则插入*/

Status InsertHash(HashTable *H, int key, int *p, int *c)

{ if (SearchHash(*H, key, p, c))//找不到插入新值

return ERROR;

if (*c < H->length)//控制冲突次数

{ _____= key;

_____++;

***c = 0;//重新初始化**

return OK;

}

else

{ *c = 0; return ERROR; }

}

Experiment 9-1 哈希表实现

Status InitHash(HashTable *H);

int Hash(int key); //哈希函数

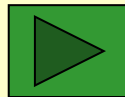
int Collision(int p, int c);

Status SearchHash(HashTable H, int key, int *p, int *c);

Status InsertHash(HashTable *H, int key, int *p, int *c);

- **哈希查找分析**

- **哈希查找过程仍是一个给定值与关键字进行比较的过程**
- **评价哈希查找效率仍要用ASL**
- **哈希查找过程与给定值进行比较的关键字的个数取决于：**
 - » **哈希函数**
 - » **处理冲突的方法**
 - » **哈希表的填满因子 α =表中填入的记录数/哈希表长度**



处理冲突的方法	平均查找长度	
	成功	不成功
线性探测再散列	$S_{nl} \approx \frac{1}{2} \left(1 + \frac{1}{1-\alpha}\right)$	$U_{nl} \approx \frac{1}{2} \left(1 + \frac{1}{(1-\alpha)^2}\right)$
随机、二次、再 哈希探测再散列	$S_{nr} \approx -\frac{1}{\alpha} \ln(1-\alpha)$	$U_{nr} \approx \frac{1}{1-\alpha}$
链地址法	$S_{nc} \approx 1 + \frac{\alpha}{2}$	$U_{nc} \approx \alpha + e^{-\alpha}$

已知一组关键字(19,14,23,1,68,20,84,27,55,11,10,79)。

哈希函数为: $H(\text{key}) = \text{key} \text{ MOD } 13$, 表长为 $m=16$, 设每个记录的查找概率相等

(1) 用线性探测再散列处理冲突, 即 $H_i = (H(\text{key}) + d_i) \text{ MOD } m$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	14	1	68	27	55	19	20	84	79	23	11	10			

$H(19)=6$

$H(11)=11$

$H(14)=1$

$H(10)=10$ 冲突, $H_1=(10+1)\text{MOD}16=11$

$H(23)=10$

冲突, $H_2=(10+2)\text{MOD}16=12$

$H(1)=1$ 冲突, $H_1=(1+1)\text{MOD}16=2$

$H(79)=1$ 冲突, $H_1=(1+1)\text{MOD}16=2$

$H(68)=3$

冲突, $H_2=(1+2)\text{MOD}16=3$

$H(20)=7$

冲突, $H_3=(1+3)\text{MOD}16=4$

$H(84)=6$ 冲突, $H_1=(6+1)\text{MOD}16=7$

冲突, $H_4=(1+4)\text{MOD}16=5$

冲突, $H_2=(6+2)\text{MOD}16=8$

冲突, $H_5=(1+5)\text{MOD}16=6$

$H(27)=1$ 冲突, $H_1=(1+1)\text{MOD}16=2$

冲突, $H_6=(1+6)\text{MOD}16=7$

冲突, $H_2=(1+2)\text{MOD}16=3$

冲突, $H_7=(1+7)\text{MOD}16=8$

冲突, $H_3=(1+3)\text{MOD}16=4$

冲突, $H_8=(1+8)\text{MOD}16=9$

$H(55)=3$ 冲突, $H_1=(3+1)\text{MOD}16=4$

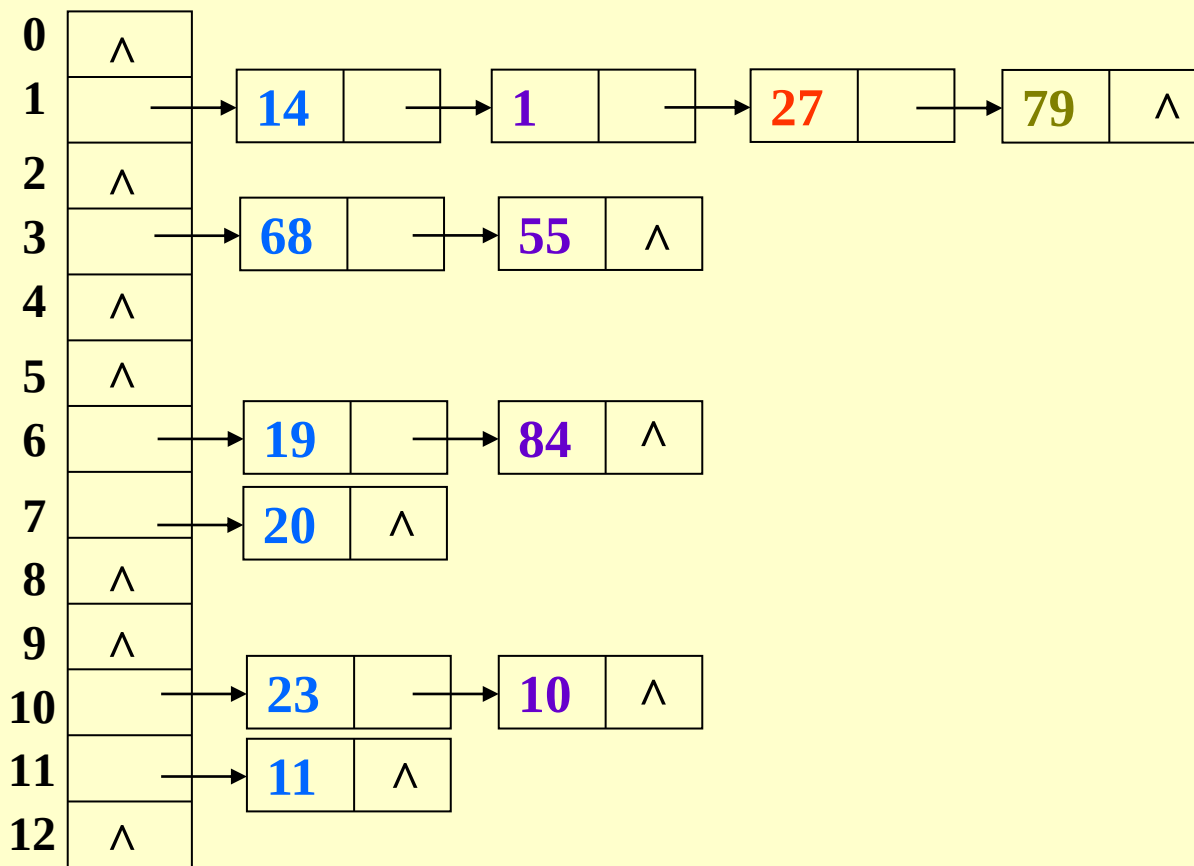
冲突, $H_2=(3+2)\text{MOD}16=5$

$ASL_{\text{成功}} = (6*1 + 1*2 + 3*3 + 1*4 + 1*9) / 12 = 2.5$

$ASL_{\text{失败}} = (1+13+12+11+10+9+8+7+6+5+4 + 3+2+1+1+1) / 16 = 94/16$

关键字(19,14,23,1,68,20,84,27,55,11,10,79)

(2) 用链地址法处理冲突



$$ASL_{\text{成功}} = (6*1 + 4*2 + 1*3 + 1*4) / 12 = 1.75$$

$$ASL_{\text{失败}} = (7*1 + 2*2 + 3*3 + 1*5) / 13 = 25/13$$



练习：使用散列函数 $H(\text{key})=\text{key}\%11$ ，把一个整数值转换成散列表下标，表长为11。现要把数据{1,13,12,34,38,33,27,22}依次插入散列表。

(1)使用线性探测法来构造散列表；

(2)使用链地址法构造散列表。

试针对地这两种情况，分别确定成功所需的平均查找长度，及查找失败所需的平均查找长度。

57、【2011统考真题】为提高散列表的查找效率，可以采取的正确措施是（）。

- I. 增大装填因子
 - II. 设计冲突少的散列函数
 - III. 处理冲突时避免产生聚集现象
 - IV. 采用链地址法处理冲突易引起聚集现象
- A、仅I B、仅II C、仅I和II D、仅II和III

58、【2014统考真题】用哈希方法处理冲突时可能出现堆积现象，下列选项中，会受堆积现象直接影响的是（）。

- A、存储效率 B、散列函数 C、装填因子 D、平均查找长度

59、一组记录的关键字为{19,14,23,1,68,20,84,27,55,11,10,79}，用链地址法构造散列表，散列函数为 $H(key)=key \text{ MOD } 13$ ，散列地址为1的链中有（）个记录。

- A、1 B、2 C、3 D、4

60、设散列有长 $m=14$ ，散列函数为 $H(key)=key \text{ MOD } 11$ ，表中仅有4个结点 $H(15)=4$ ， $H(38)=5$ ， $H(61)=6$ ， $H(84)=7$ ，若采用线性探测法处理冲突，则关键字为49的结点地址是（）。

- A、8 B、3 C、5 D、9

61、【2018统考真题】现有长度为7，初始这空的散列表HT，散列函数 $H(k)=k\%7$ ，用线性探测再散列法解决冲突。将关键字22,43,15依次插入到HT后，查找成功的平均查找长度是（）。

- A、1.5 B、1.6 C、2 D、3

62、【2019统考真题】现有长度为11，初始这空的散列表HT，散列函数 $H(k)=k\%7$ ，用线性探测再散列法解决冲突。将关键字87,40,30,6,11,22,98,20依次插入到HT后，查找失败平均查找长度是（）。

- A、4 B、5.25 C、6 D、6.29